Gabriel Coutinho - rm377693

Tiago Alcântara - rm378125

Caue Ferreira - rm378843

Paulo Henrique Silva - rm377492

Rafael Broglia Szyszko - rm378642

FIAP
2026

Motivo da aplicação ser um Monolito:

Monolito se refere ao design de arquitetura, onde os componentes da aplicação estão interligados em uma única unidade. Algumas de suas características podem ser observadas por alguns fatores, como integração a uma única base de dados, uma estrutura fortemente acoplada, um único Deploy pra toda a aplicação.

Como analisado, o projeto toggle-master-monolith representa um monolito, pois ele abrange todas as características citadas anteriormente:

- **Banco de dados** ➔ Toda a aplicação está integrada a um banco Postgres;
- **Estrutura acoplada** ➔ Todos serviços estão centralizados em uma única aplicação, fazendo com que o acoplamento e a dependência entre componentes sejam altos;
- **Deploy único** ➔ Devido a característica apresentada anteriormente, consequentemente o método de Deploy será feito de forma unitária.

VANTAGENS:

Nesse contexto, a padronização de tecnologias e ferramentas seriam aplicadas de forma mais fácil, gerando um tipo de Governança centralizada, sendo mais fácil de gerenciar e organizar.

Como todos os serviços estão integrados a uma única base de dados, a consistência entre tabelas seria mais segura, já que a alteração feita um serviço instantaneamente refletiria no outro.

DESVANTAGENS:

Por tudo estar centralizado em uma única estrutura de código, o sistema precisaria ser escalado como um todo, sem ser feito por componentes de forma independente, o que pode resultar em maiores custos com infraestrutura, tempo de desenvolvimento, entre outros.

Consequentemente, a manutenção dessa aplicação se tornaria cada vez mais difícil, levando em conta que, caso você precise alterar somente uma parte do sistema, outras partes podem ser afetadas devido ao alto acoplamento.

O que no final poderia ocasionar em um Deploy mais longo e suscetível a erros, já que haveria o Build de todos os componentes de uma única vez.

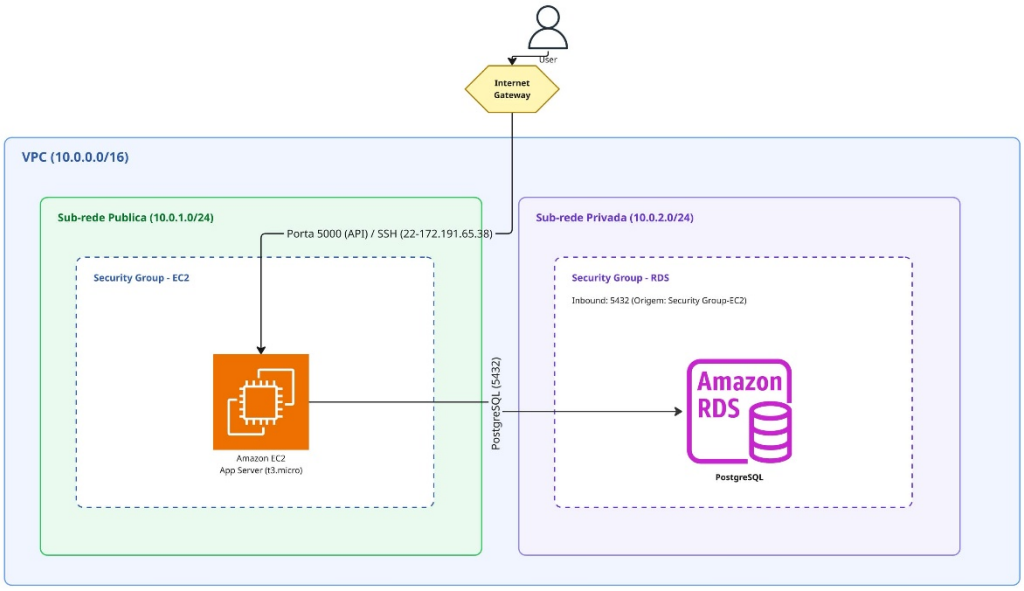
Além disso, trabalhar com Monolito em um projeto grande significa: coordenar times diferentes, onde todos fariam mudanças na mesma codebase, podendo ocorrer algum tipo de conflito de merge e falhas na comunicação; e limitações tecnológicas, onde seria mais difícil adotar novas tecnologias ou frameworks,

em que mudanças como essa significam reescrever boa parte do sistema.

Diagrama da arquitetura:

Como solicitado, abaixo segue o diagrama da arquitetura, utilizando os seguintes serviços e recursos AWS:

- **VPC** ➔ Serviço de rede privada da AWS, onde configuramos as subnets, sejam elas públicas ou privadas;
- **EC2** ➔ Serviço de computação, onde subimos as aplicações em Máquinas Virtuais;
- **RDS** ➔ Serviço de Banco de Dados relacionais;
- **Security group** ➔ Recurso de firewall que delimita quem consegue se conectar com determinados serviços.

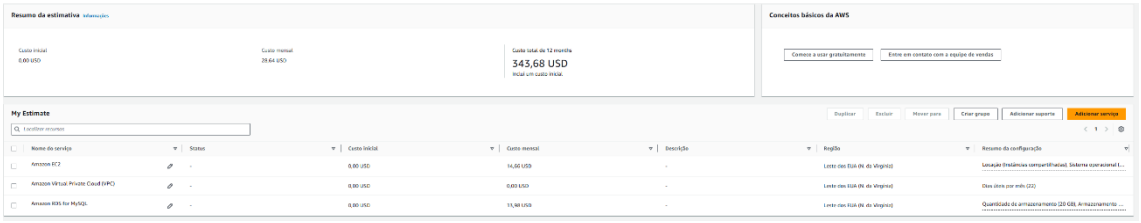


Vídeo de demonstração:

No vídeo de demonstração, disponível em https://youtu.be/a-g04HLmb8A, apresentamos a aplicação rodando localmente, a explicação do diagrama de arquitetura proposto e a demonstração da aplicação rodando na EC2, acessando o banco de dados no RDS. Também mostramos as configurações de Security Group e como as credenciais do banco foram gerenciadas.

Precificação dos serviços AWS:

O preço atribuído aos serviços mostrados anteriormente, representam o uso constante e permanente da aplicação. Abaixo há uma estimativa sendo apresentada por mês e ano:



Checklist 12-Factor App

Análise de conformidade do projeto toggle-master-monolith (ToggleMaster — Tech Challenge Fase 1) com a metodologia The Twelve-Factor App.

Repositório	toggle-master-monolith — branch `main`, commit `0478ad5`
Data da análise	06/10/2026
Arquivos analisados	`app.py`, `Dockerfile`, `docker-compose.yaml`, `entrypoint.sh`, `requirements.txt`, `README.md`
Stack	Python 3.9 · Flask 2.2.2 · Gunicorn 20.1.0 · psycopg2 · PostgreSQL 13 (local) / Amazon RDS (produção) · EC2

1. Metodologia

Cada um dos 12 fatores foi avaliado com base no que está de fato no código e na documentação do repositório, considerando os dois ambientes descritos pelo projeto: o ambiente local (Docker Compose) e o ambiente de produção (deploy manual em EC2 + RDS, conforme o `README.md`). Cada fator recebeu um de três status:

Status	Critério
ATENDE	O princípio é seguido; podem existir apenas ressalvas menores.
ATENDE PARCIALMENTE	Parte do princípio é seguida, mas há lacunas concretas que podem causar problemas.
NÃO ATENDE	O fluxo atual contraria o princípio.

2. Resumo

8	**3**	**1**
Atendem	Atendem parcialmente	Não atende

#	Fator	Status	Motivo principal
I	Base de código	**ATENDE**	Um único repositório Git para um único app; vários deploys a partir dele.
II	Dependências	**ATENDE**	Dependências diretas declaradas com versão fixa no `requirements.txt` e isoladas em contêiner/`venv`.

#	Fator	Status	Motivo principal
III	Configurações	**ATENDE**	Credenciais e dados de conexão do banco lidos de variáveis de ambiente; nada sensível no código.
IV	Serviços de apoio	**ATENDE**	PostgreSQL local e RDS são trocados só por configuração, sem mudar código.
V	Build, release, run	**NÃO ATENDE**	Em produção o build acontece no próprio servidor (`git clone` + `pip install`); não há releases versionadas.
VI	Processos	**ATENDE**	App stateless; todo estado fica no PostgreSQL.
VII	Vínculo de porta	**ATENDE**	Gunicorn embutido expõe HTTP na porta 5000; não depende de servidor web externo.
VIII	Concorrência	**PARCIAL**	Arquitetura permite escalar horizontalmente, mas roda com 1 worker padrão e sem gerenciador de processos.
IX	Descartabilidade	**PARCIAL**	Inicialização rápida e `exec` para receber SIGTERM, mas em produção o processo morre ao fechar o SSH e não é reiniciado.
X	Paridade dev/prod	**PARCIAL**	Mesmo tipo de banco, mas local roda em contêiner e produção roda direto no SO, com outra versão do Python e sem o `entrypoint.sh`.
XI	Logs	**ATENDE**	Saída em stdout/stderr sem buffer; nenhum arquivo de log é gerenciado pelo app.
XII	Processos de administração	**ATENDE**	`flask init-db` roda como processo pontual, com o mesmo código e a mesma config.

> **Achado crítico (fatores V, X e XII):** seguindo o guia de deploy na EC2 do `README.md`, a tabela `flags` nunca é criada. A criação só acontece em `flask init-db`, que é chamado pelo `entrypoint.sh` (usado apenas no Docker). Na EC2 o app é iniciado direto com `gunicorn`, então o primeiro `POST /flags` retorna erro 500 (`relation "flags" does not exist`). É preciso rodar `flask init-db` manualmente ou incluir esse passo no guia.

3. Análise detalhada por fator

I. Base de código (Codebase) — ATENDE

Uma base de código com controle de versão, vários deploys.

EVIDÊNCIAS:

- Todo o app está em um único repositório Git (`app.py`, `Dockerfile`, `docker-compose.yaml`, `entrypoint.sh`, `requirements.txt`).
- O mesmo código gera os dois deploys previstos: local (Docker Compose) e produção (EC2 via `git clone`).
- Não há código compartilhado copiado entre repositórios nem vários apps no mesmo repositório.

POR QUE ATENDE:

Existe exatamente uma relação 1:1 entre a base de código e o app, e os ambientes diferem só na configuração, não no código.

RESSALVAS:

- Faltam `.gitignore` e `.dockerignore`: o `venv/` criado na EC2 e o diretório `.git` podem acabar versionados ou copiados para a imagem.
- Não há tags/versões no Git para identificar qual commit está em cada deploy.

II. Dependências — ATENDE

Declare e isole as dependências explicitamente.

EVIDÊNCIAS:

- `requirements.txt` declara as dependências diretas com versão fixa (`Flask==2.2.2`, `Werkzeug==2.3.8`, `psycopg2-binary==2.9.5`, `gunicorn==20.1.0`).
- Isolamento: a imagem Docker (`python:3.9-slim`) e o `venv` na EC2 evitam depender de pacotes do sistema.
- A ferramenta de sistema `pg_isready` é declarada explicitamente no Dockerfile (`apt-get install postgresql-client`), em vez de supor que ela já exista.

POR QUE ATENDE:

As dependências do app são declaradas em um manifesto versionado junto com o código e instaladas em um ambiente isolado, tanto localmente quanto em produção, sem depender de pacotes pré-existentes no sistema.

III. Configurações (Config) — ATENDE

Armazene as configurações no ambiente.

EVIDÊNCIAS:

- `app.py:9-12` lê `DB_HOST`, `DB_NAME`, `DB_USER` e `DB_PASSWORD` com `os.getenv`; nenhuma credencial de produção está no código.
- O `README.md` orienta a exportar as credenciais do RDS como variáveis de ambiente e avisa para nunca versioná-las.
- O `entrypoint.sh` valida a presença das variáveis antes de iniciar (fail-fast).

POR QUE ATENDE:

Tudo o que muda entre os ambientes (host, banco, usuário e senha) é lido de variáveis de ambiente. O mesmo código roda localmente e na EC2 + RDS apenas trocando os valores dessas variáveis.

IV. Serviços de apoio (Backing services) — ATENDE

Trate serviços de apoio como recursos anexados.

EVIDÊNCIAS:

- O PostgreSQL é acessado só pela rede, por meio de host/usuário/senha vindos do ambiente (`get_db_connection()`, `app.py:14-21`).
- O mesmo código conversa com o contêiner `db` localmente e com o Amazon RDS em produção. A troca é feita apenas mudando `DB_HOST` etc., sem alterar código.

POR QUE ATENDE:

O banco é um recurso anexado e substituível: não há diferença no código entre um banco local e um banco gerenciado de terceiros.

RESSALVA:

A porta não é configurável, o que limita um pouco essa troca.

V. Build, release, run — NÃO ATENDE

Separe estritamente os estágios de build e execução.

O QUE EXISTE:

O Dockerfile define um estágio de build reproduzível, o que é uma boa base.

POR QUE NÃO ATENDE:

- No fluxo de produção do `README.md`, o build acontece no próprio servidor de execução: `git clone` ➔ `pip install` ➔ `export` ➔ `gunicorn`, tudo manual na EC2. Não há artefato imutável.

- Não existe estágio de release: nenhum identificador de versão (tag, número de release ou tag de imagem), nenhum registry (ex.: ECR) e nenhuma forma de rollback para uma release anterior.
- O `docker-compose.yaml` monta `.:/app` como volume, o que sobrescreve em tempo de execução o código que foi incluído na imagem. É útil para desenvolvimento, mas mistura os estágios.
- Não há pipeline de CI/CD.

RECOMENDAÇÃO:

Construir a imagem no CI, publicá-la no ECR com tag do commit/versão (build), combinar com a config do ambiente (release) e apenas executar a imagem na EC2/ECS (run).

VI. Processos **ATENDE**

Execute o app como um ou mais processos sem estado.

EVIDÊNCIAS:

- Nenhum estado é mantido em memória entre requisições: não há sessão, cache local nem variáveis globais mutáveis.
- Nenhum arquivo é gravado no disco local; todo dado persistente (as flags) fica no PostgreSQL. Cada requisição abre e fecha sua própria conexão com o banco.

POR QUE ATENDE:

Qualquer instância do processo pode atender qualquer requisição, e matar uma instância não perde dados.

RESSALVA:

Abrir uma conexão nova por requisição é stateless, mas pouco eficiente. Um pool (ex.: `psycopg2.pool`) ou RDS Proxy melhoraria o desempenho sem quebrar o fator.

VII. Vínculo de porta (Port binding) **ATENDE**

Exporte serviços via vínculo de porta.

EVIDÊNCIAS:

- O app é autocontido: o Gunicorn, declarado como dependência, sobe o servidor HTTP e faz bind em `0.0.0.0:5000` (`entrypoint.sh:37`).
- Não depende de Apache/Nginx ou de outro servidor injetado em tempo de execução; o Dockerfile só faz `EXPOSE 5000`.

RESSALVA:

A porta é fixa. O ideal é ler de uma variável (`--bind 0.0.0.0:${PORT:-5000}`).

VIII. Concorrência — ATENDE PARCIALMENTE

Escale horizontalmente através do modelo de processos.

O QUE ESTÁ PRESENTE:

- Como o app é stateless (fator VI), é possível rodar várias instâncias atrás de um load balancer (ALB) ou várias réplicas no Compose.
- O Gunicorn segue o modelo de processos (master + workers).

O QUE FALTA:

- O Gunicorn é iniciado sem `--workers`, ou seja, com o padrão de 1 worker síncrono: uma requisição por vez.
- Não há formação de processos declarada (Procfile, réplicas no Compose, Auto Scaling Group) nem gerenciador de processos em produção. O próprio `README.md` diz que o app roda em foreground, sem `systemd`.
- O `ports: "5000:5000"` fixo no Compose impede subir mais de uma réplica no mesmo host.

RECOMENDAÇÃO:

Configurar `--workers` via variável (ex.: `WEB_CONCURRENCY`), gerenciar o processo com `systemd` ou ECS e escalar horizontalmente com ASG/ECS Service.

IX. Descartabilidade (Disposability) — ATENDE PARCIALMENTE

Maximize a robustez com inicialização rápida e desligamento gracioso.

O QUE ESTÁ PRESENTE:

- A inicialização é rápida: o app é pequeno e não tem carga pesada no boot.
- `entrypoint.sh:37` usa `exec gunicorn`, então o Gunicorn vira o PID 1 e recebe o SIGTERM do `docker stop`, fazendo shutdown gracioso.
- O loop com `pg_isready` evita falhas se o banco ainda não estiver pronto, e o `depends_on: service_healthy` coordena a ordem no Compose.

O QUE FALTA:

- Em produção (EC2) o processo roda em foreground na sessão SSH: morre ao desconectar e não é reiniciado após falha ou reboot.
- Não há `restart:` policy no `docker-compose.yaml`.
- O loop do `pg_isready` não tem timeout: se o banco nunca subir, o contêiner fica preso para sempre em vez de falhar e ser substituído.
- O Dockerfile define um `CMD` que é ignorado, porque o `entrypoint.sh` executa um comando fixo em vez de `exec "$@"`.

RECOMENDAÇÃO:

Usar `systemd` (`Restart=always`) ou contêiner com restart policy em produção, adicionar timeout ao loop de espera e trocar a última linha do entrypoint por `exec "$@"`.

X. Paridade entre desenvolvimento e produção — ATENDE PARCIALMENTE

Mantenha desenvolvimento, homologação e produção o mais semelhantes possível.

O QUE ESTÁ PRESENTE:

- O mesmo tipo de serviço de apoio é usado nos dois ambientes (PostgreSQL), sem trocar por SQLite em dev.
- O Docker Compose permite que qualquer desenvolvedor suba um ambiente idêntico ao dos colegas.

O QUE FALTA:

- Lacuna de ferramentas: local roda em contêiner (`python:3.9-slim`); produção roda direto no SO da EC2, com o Python da distribuição (3.9 a 3.12, conforme Amazon Linux ou Ubuntu).
- Processo de inicialização diferente: local passa pelo `entrypoint.sh` (espera o banco + `init-db`); produção chama o `gunicorn` direto, por isso a tabela não é criada (ver achado crítico).
- A versão do PostgreSQL é fixa em 13 localmente, mas não há indicação de qual versão usar no RDS.
- Lacuna de tempo/pessoal: deploy manual via SSH, sem CI/CD, o que aumenta o tempo entre o código escrito e o código em produção.

RECOMENDAÇÃO:

Rodar a mesma imagem Docker em produção (EC2 com Docker ou ECS), fixar a mesma versão maior do PostgreSQL no RDS e automatizar o deploy.

XI. Logs — ATENDE

Trate logs como fluxos de eventos.

EVIDÊNCIAS:

- O app escreve em stdout com `print()` e o Gunicorn escreve em stderr; nenhum arquivo de log é aberto, rotacionado ou gerenciado pelo app.
- `PYTHONUNBUFFERED=1` no Dockerfile garante que os logs saiam imediatamente, prontos para `docker logs`, journald ou CloudWatch.

POR QUE ATENDE:

O app não se preocupa com roteamento nem armazenamento de logs; isso fica a cargo do ambiente de execução.

RESSALVAS:

- Usar o módulo `logging` (com nível e, de preferência, formato JSON) em vez de `print()`. O access log do Gunicorn está desativado por padrão; adicionar `--access-logfile -`.
- Os erros 500 devolvem `str(e)` ao cliente (`"details"`). Esse detalhe deveria ir para o log, não para a resposta, pois pode expor informações internas.

XII. Processos de administração — ATENDE

Execute tarefas de administração/gestão como processos pontuais.

EVIDÊNCIAS:

- A criação do schema é um comando CLI pontual, `flask init-db` (`app.py:45-47`), que fica na base de código e usa as mesmas dependências e a mesma config do app.
- O comando é idempotente (`CREATE TABLE IF NOT EXISTS`), então pode ser executado de novo com segurança.

POR QUE ATENDE:

A tarefa administrativa é entregue junto com o app e roda no mesmo ambiente, em vez de ser um script SQL manual solto.

RESSALVAS:

- Os erros são engolidos duas vezes: `init_db()` só faz print da exceção (exit code 0) e o entrypoint ainda usa `|| true`. Uma falha de migração passa despercebida.
- O guia de EC2 não manda executar `flask init-db`.
- Para evoluir o schema, o ideal é uma ferramenta de migração versionada (ex.: Alembic / Flask-Migrate).

4. Plano de ação priorizado

#	Ação	Fatores	Esforço
1	Incluir `flask init-db` no guia de deploy da EC2 (ou rodar o app via Docker na EC2).	X, XII	Baixo
2	Criar `.gitignore` / `.dockerignore`.	I	Baixo
3	Rodar o Gunicorn sob `systemd` (ou contêiner com restart policy) e com `--workers` configurável.	VIII, IX	Baixo
4	Trocar `print` por `logging`, ativar access log e parar de expor `str(e)` nas respostas.	XI	Baixo
5	Pipeline de CI que constrói e publica a imagem no ECR com tag versionada; produção apenas executa a imagem.	V, X	Médio

Infraestrutura como Código (Terraform):

Para facilitar a subida da infraestrutura na AWS, adicionamos o Terraform ao projeto (https://github.com/rafaelszyszko/entrega-1-devops-fiap-postech). Com ele, os recursos utilizados na arquitetura, como VPC, sub-redes, EC2, RDS e Security Groups, são criados a partir de código, em vez de serem configurados manualmente pelo console da AWS.

Além de agilizar a entrega, essa decisão foi uma oportunidade para o grupo aprender Infraestrutura como Código (IaC) com um integrante que já tinha experiência com a ferramenta.

Desafios encontrados e decisões tomadas:

As principais dificuldades enfrentadas pelo grupo foram:

- **12-Factor App** ➔ Decidir o status de cada um dos 12 fatores gerou discussão dentro do grupo até chegarmos a um consenso sobre a classificação de cada um;
- **Plano de ação** ➔ Determinar quais ações priorizar e o esforço de cada uma, a partir das lacunas encontradas na análise;
- **Terraform** ➔ Como parte do grupo ainda não conhecia IaC, estruturar e entender o código Terraform também foi um desafio, superado com o apoio de quem já dominava a ferramenta.

Como decisões, o grupo definiu a classificação dos fatores apresentada no checklist, o plano de ação priorizado e a adoção do Terraform para provisionar a infraestrutura.

Repositório GitHub:

O código-fonte do projeto está disponível no repositório:
https://github.com/TiagoAlcan/fase1_devops_fiap_postech.git